Agent and Codex Development Workflow

Purpose

This document defines how Codex or other coding agents scope work, create branches, validate changes, open pull requests and participate in review for FinWallet. Repository files, AGENTS.md, documentation, ADRs and GitHub issues/PRs are the source of truth; chat history does not replace repository documentation.

Operating model

Each agent:

1. receives a bounded task/issue;
2. reads repository rules;
3. creates an agent/<task> branch from current main;
4. makes required code/test/document changes;
5. validates with Release build/tests;
6. opens a reviewable pull request.

Recommended roles:

- Solution Architect;
- Security/Auth;
- Financial Domain;
- Persistence/Concurrency;
- Integration;
- QA/Chaos;
- Code Review;
- Documentation.

Source-of-truth order

When instructions conflict:

1. current task acceptance criteria;
2. AGENTS.md;
3. master specification;
4. accepted ADR;
5. architecture/security/database/API docs;
6. existing implementation conventions;
7. agent assumptions.

An agent must not silently override a higher-level rule.

Branch/commit

Branch format: agent/<bounded-task-name>.

Commits should be small and cohesive. Avoid vague commits such as implement everything or fix stuff. Financial behavior, package changes, refactors and documentation should remain independently

understandable when practical.

Coding-agent responsibilities

- preserve financial invariants;
- write required TR/EN XML docs for classes/interfaces/methods/properties;
- do not add paid/freemium dependencies;
- update package inventory for new packages;
- add/update tests for changed behavior;
- update API/architecture/database/security docs;
- never place external HTTP inside a financial SQL transaction;
- preserve idempotency/concurrency guarantees;
- never make Redis the financial source of truth;
- keep TR+EN Markdown documentation synchronized.

Review-agent checks

- architecture dependency violations;
- ledger/financial correctness;
- double-spend/lost updates;
- idempotency gaps;
- auth/OTP/token weaknesses;
- sensitive logging;
- unsafe retry/timeout behavior;
- missing cancellation;
- over-engineering;
- undocumented packages;
- missing TR/EN code/documentation.

QA/Chaos scenarios

- simultaneous overspend;
- repeated idempotency key;
- same key + altered payload;
- Redis outage;
- provider timeout/500/slow response;
- fraud unavailable;
- repeated refund/reversal;
- ledger mismatch;
- refresh-token replay;
- OTP brute force;
- direct Gateway bypass;
- rate-limit/resource exhaustion.

Pull-request flow

1. Complete bounded work on the branch.
2. Self-review the diff.
3. Update docs/tests.
4. Run dotnet restore, Release dotnet build --warnaserror, and dotnet test.
5. Open a draft PR.
6. Resolve review findings with small commits.
7. CI becomes green.
8. Financial/security-sensitive PRs receive final review.
9. Merge only after acceptance criteria are satisfied.

Parallel-agent rule

Until domain boundaries, DB/ledger rules, API contracts, auth and error/idempotency conventions are stable, parallel agents should not invent incompatible contracts inside the same feature area. Parallel work is appropriate when file overlap is low and contracts are stable.

DoD

Agent-generated code is done only when acceptance criteria, build, relevant tests, financial/security invariants, TR/EN XML docs, package inventory and affected TR/EN Markdown docs are complete.